Network Telemetry Framework

Real-Time Network Performance Monitoring | RFC 9232

CS331 Computer Networks | Final Project Presentation

Avinash Reddy
23110123

Gella Jayarama Krishna
23110115

Sowpati Raj Kamal
23110319

Jangam Sanjay
23110144

Pulakurthi Manohar
23110259

Yalla Sai Teja
23110366

Department of Computer Science & Engineering

Project Motivation & Technical Approach

Limitations of periodic polling vs. continuous push-based streaming

The Challenge: Periodic SNMP Polling

- Traditional monitoring polls network devices on fixed multi-minute intervals (e.g., 5 min).
- Transient congestion bursts and micro-spikes are smoothed out in long averages.
- Centralized polling introduces CPU load and management overhead as network size scales.

The Solution: Continuous Push Telemetry (RFC 9232)

- Edge devices stream performance measurements proactively over persistent gRPC channels.
- High-frequency telemetry generation (2 s interval) exposes transient anomalies in seconds.
- Push model eliminates polling latency and reduces device CPU overhead.
- Operates continuously, transporting compact binary Protocol Buffer payloads.

Target Performance Metrics:

1. Bandwidth Utilization: Real-time link capacity consumption (0% to 100%)
2. Throughput Rates: Ingress and egress bitrates in Mbit/s
3. Round-Trip Latency: End-to-end TCP connection response time in milliseconds
4. Interface Health: Monitoring packet errors, discards, and dropped frames

Tools & Core Technologies

- **Python 3.12:** Core runtime powering backend services, agents, and test harness.
- **gRPC & Protobuf:** High-performance streaming transport encoding metrics in 72-byte payloads.
- **FastAPI & Uvicorn:** Asynchronous REST backend serving real-time API queries and dashboard.
- **SQLite Database:** Embedded persistence layer with Write-Ahead Logging (WAL) mode.
- **psutil:** Operating-system interface sampling physical host network statistics.
- **HTML5 Canvas:** Hardware-accelerated browser rendering for real-time time-series charts.

Key Project Deliverables

- 1. Distributed edge agents supporting physical host and simulated network nodes
- 2. Asynchronous backend ingestion pipeline with decoupled storage queue
- 3. Threshold-based alerting engine with anti-flapping hysteresis logic
- 4. Real-time web dashboard, REST query APIs, and CSV/JSON export capability
- 5. Automated test suite with 30 passing unit and integration test cases

System Architecture & Functional Pipeline

End-to-end telemetry architecture mapping to RFC 9232

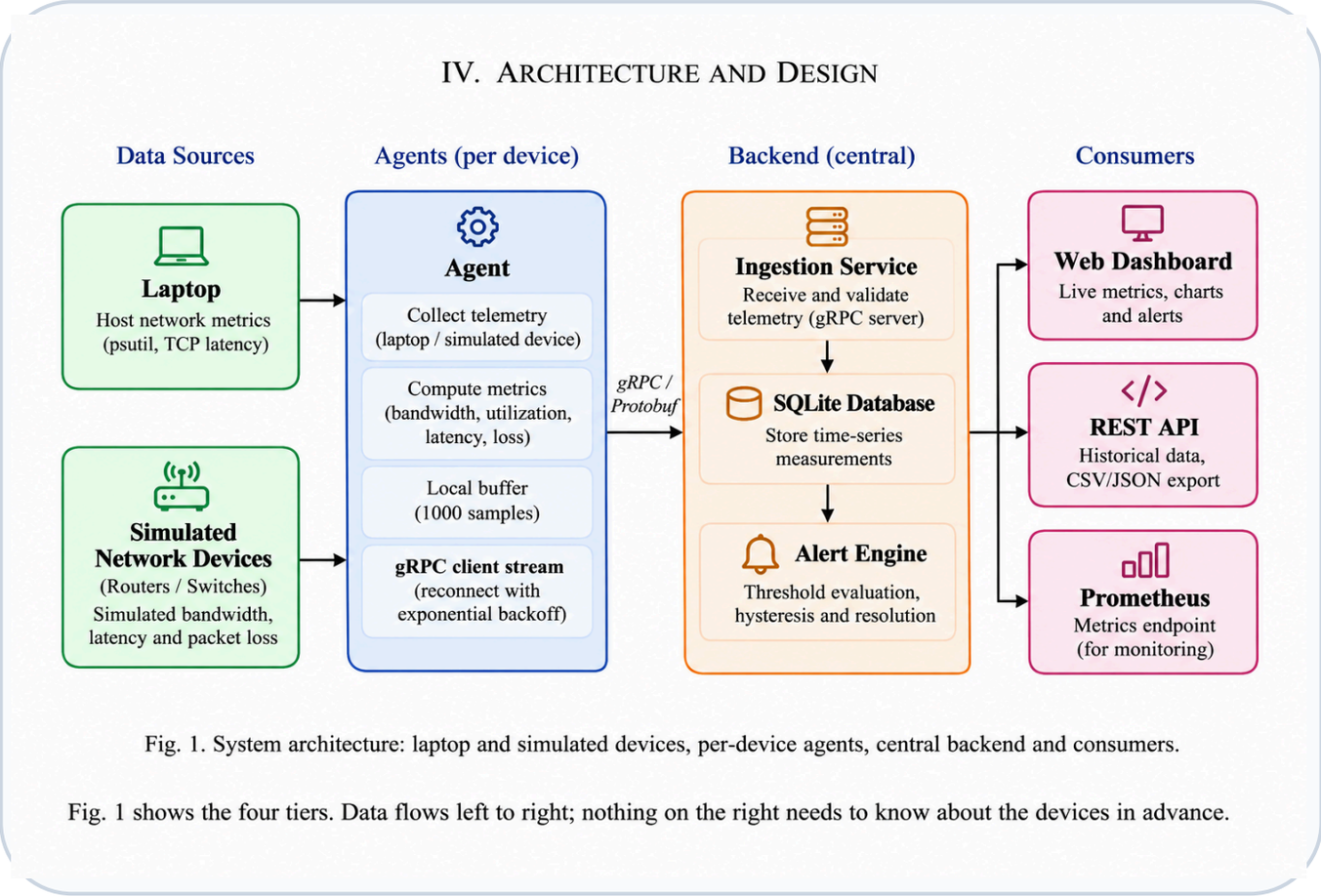


Fig. 1. System architecture: laptop and simulated devices, per-device agents, central backend and consumers.

Fig. 1 shows the four tiers. Data flows left to right; nothing on the right needs to know about the devices in advance.

Figure 1: End-to-end system architecture from project report

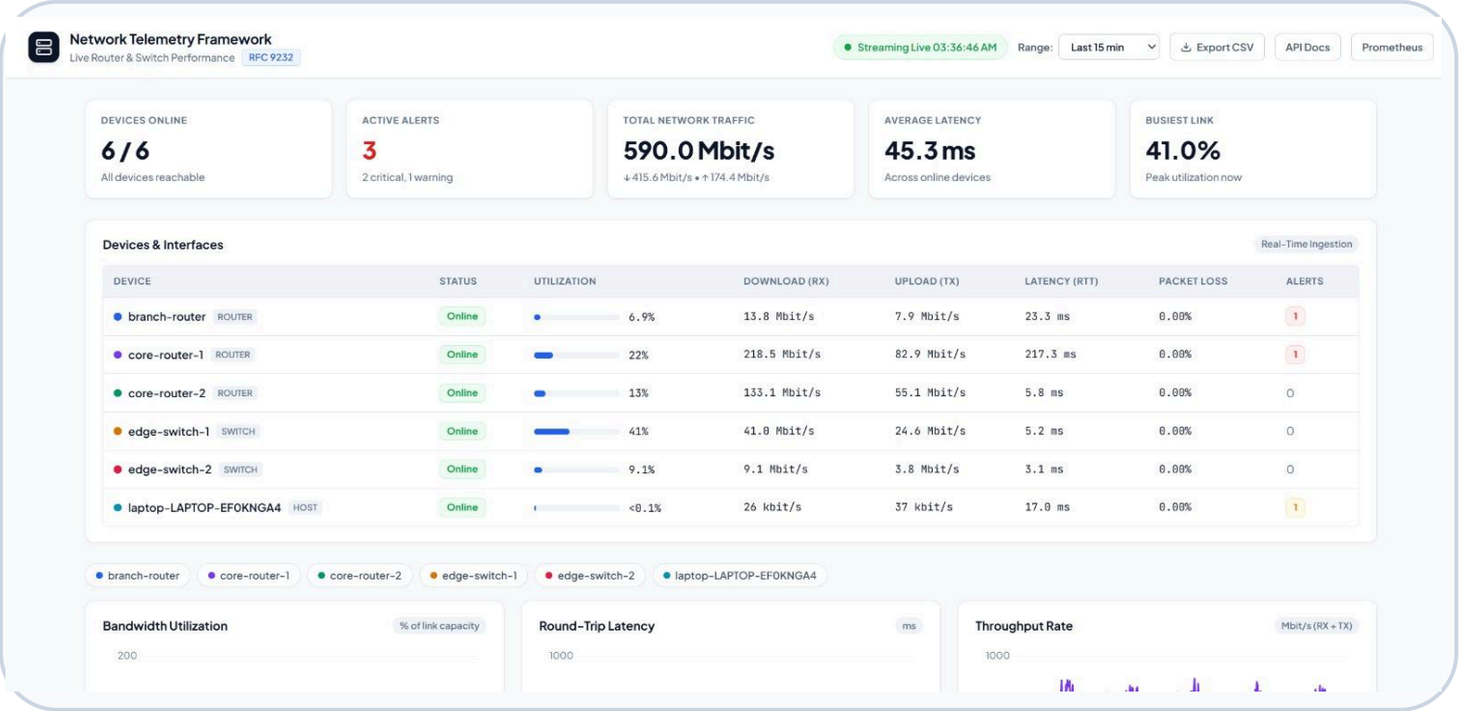
Architectural Highlights:

- **1. Data Collection Layer:**
  Dual-collector design: physical host network stats via psutil and TCP latency probing, combined with 5 simulated routers and switches modeling enterprise loads.
- **2. Transport Layer (gRPC):**
  Persistent HTTP/2 client stream (Telemetry.Stream). Telemetry messages are encoded in compact Protocol Buffers (~72 bytes) and streamed at 2-second intervals.
- **3. Ingestion & Storage Layer:**
  An in-memory queue (capacity 20,000) decouples network reception from disk. SQLite persists telemetry in 500-record batches using Write-Ahead Logging (WAL).
- **4. Alerting & Visualization Layer:**
  Hysteresis state machine evaluates thresholds, triggering alerts after 2 breach cycles. Real-time HTML5 canvas dashboard and REST APIs expose telemetry.

RFC 9232 Mapping: Data Generation -> Push Export -> Storage -> Analysis & Visualization

Real-Time Telemetry Dashboard

Live performance visualization, link capacity tracking, and incident management



Live Web Dashboard running at http://localhost:8000

Dashboard Capabilities:

- **KPI Health Overview:**
  Instant summary cards displaying reachable device count (6/6), active alert counts, total throughput (1.65 Gbit/s), and average latency.
- **Interface Utilization Table:**
  Tabular monitoring of routers, switches, and hosts with live bitrates, round-trip latency, packet loss, and animated utilization gauges.
- **Dynamic Sub-0.1% Precision:**
  Correctly preserves sub-0.1% utilization on idle links rather than rounding down to zero, maintaining telemetry continuity.
- **Canvas Trend Visualizations:**
  Hardware-accelerated HTML5 canvas line graphs plotting Bandwidth Usage, Latency, and Throughput with smooth 2-second updates.
- **Historical Data Export:**
  One-click CSV export engine generating comprehensive, audit-ready telemetry logs for offline analysis in under 0.6 seconds.

Experimental Testbed Topology:

- 1 Physical Host Node: Live OS network counters sampled via psutil with active TCP connection probing
- 5 Simulated Network Nodes: 2 core routers, 2 edge switches, 1 branch router with scripted congestion cycles
- Update Cadence: 1-second continuous gRPC push stream; dashboard refreshes every 2 seconds via REST polling

Experimental Evaluation & Results

Empirical performance benchmarks from the project report (Section VII)

400 Devices

0 Dropped Packets

Tested with 400 concurrent simulated devices at 1 sample/s with zero data loss.

2.8 Seconds

Mean Alert Time

Injected congestion incidents were detected and alerted in an average of 2.8 seconds.

71% Reduction

Flapping Suppression

Hysteresis state machine eliminated transient noise, cutting false alerts from 101/hr to 29.6/hr.

30 / 30 Passed

Automated Tests

100% test pass rate across Protobuf schemas, database queues, alert logic, and REST APIs.

Experiment E1: Scalability & Throughput (Up to 400 Devices)

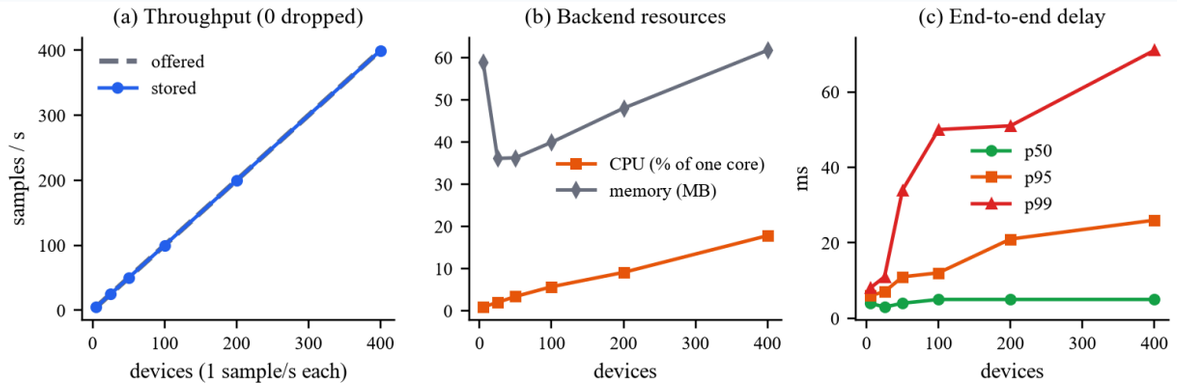


Figure 2: Offered vs stored throughput, CPU & memory usage, and p95 latency

Experiment E3: Congestion Alert Detection & Recovery



Figure 4: Injected incidents on edge-switch-1 with alert opening windows

Summary of Experimental Findings:

1. High Throughput & Zero Loss: The decoupled queue architecture sustains 400 samples/s with zero dropped packets and p95 latency under 26 ms.
2. Alert Reliability: Hysteresis evaluation accurately isolates sustained congestion from transient noise, resolving automatically post-recovery.

Summary & Conclusions

- **Delivered Pipeline:**
  A modular network telemetry framework implementing RFC 9232 principles to collect, transport, store, and visualize performance metrics in real time.
- **High-Speed Streaming:**
  Replaces legacy SNMP pull polling with lightweight gRPC HTTP/2 push streaming using compact Protocol Buffers (~72 bytes/sample).
- **Empirically Validated:**
  Validated through 30 automated Pytest tests, 400-device scalability benchmarking, and scripted fault-injection scenarios confirming zero data drops.
- **Scope & System Boundaries:**
  Implemented as an educational prototype using physical laptop OS metrics (psutil) and simulated enterprise switches/routers without proprietary ASIC hardware.

Future Work:

- Integration with physical network switching hardware (Cisco / Juniper)
- Standard gNMI (gRPC Network Management Interface) schema support
- Mutual TLS (mTLS) transport encryption and token authentication
- Scalable time-series storage backend (TimescaleDB / InfluxDB)

Core Architectural Contribution:

Demonstrates high-resolution, sub-second network observability via push streaming.

Thank You!

Questions & Discussion

Project Code & Reproducibility

https://github.com/Rajkamal065/network-telemetry

Automated Test Suite: python -m pytest tests/ -v (30/30 passed)

Team T018 | CS331 Computer Networks

Avinash Reddy
23110123

Gella Jayarama Krishna
23110115

Sowpati Raj Kamal
23110319

Jangam Sanjay
23110144

Pulakurthi Manohar
23110259

Yalla Sai Teja
23110366

Department of Computer Science & Engineering